

Pavel Ovchinnikov

Senior Software Engineer | Full-Stack · Backend · IoT · Cloud · AI

BASED IN Georgia (PE) **ENGLISH** B2 **EDUCATION** B.S. Computer Science

Senior Software Engineer with 10+ years of experience building production web applications, backend services, PWA/mobile, IoT platforms, and high load systems. Experienced across the full software lifecycle, from system architecture and APIs to frontend applications, databases, cloud infrastructure, and CI/CD.

My background includes frontend, backend and mobile frameworks, API and communication patterns, relational and NoSQL databases, ORMs, cloud platforms and managed services, containerization and orchestration, CI/CD and infrastructure automation, authentication and authorization systems, caching and messaging, testing, monitoring, and observability. I have worked on both product and enterprise systems, including complex data platforms and applications handling large datasets.

Currently focused on applying this engineering experience to AI-powered products and modern software systems, combining established software engineering practices with emerging AI technologies.

Mentors and leads developers formally and informally. Reviews teammates' code to catch bugs and keep quality high. Writes clear docs, branching policies, and linting rules. Good at detecting issues early using the terminal, curl, logs, and native database queries.

2026.05 - present

Meshintex, Inc.

IoT device management platform

React · TypeScript · Python · Node.js · PostgreSQL · Next.js · AWS · Mapbox · MCP · Claude API · Sentry · CI/CD · Git

At Meshintex, I built a dashboard that works for any kind of IoT setup, with a backend API that takes in live sensor data and designed the data-processing architecture to scale to thousands of connected sensors using asynchronous queues and background processing (**SNS/SQS** + **Lambda**). It shows real-time charts and maps with lots of markers on them using **MapBox**. The API and dashboard are highly customizable and scalable supporting different sensor configurations. Security was a priority the whole way through.

Implemented an **MCP**-based AI service for automated **CI/CD** error resolution. The service receives **Sentry** errors and **CI/CD** test failures, uses the **Claude AI API** to analyze errors, retrieves relevant

application documentation from **Confluence**, and attempts to implement a fix in a dedicated **Git** branch. Also MCP creates **Jira** tickets with the error description, proposed fix, and links to the branch and documentation. Implemented configurable retry limits and cost controls to prevent excessive AI API usage and runaway automated attempts. Once the pipeline succeeds, the changes are pushed to the branch for developer review, keeping humans in the loop before merging.

> Kept the map view smooth with large numbers of sensors on screen, measured by lag-free pan and zoom, by implementing marker clustering. ↑ PERFORMANCE

> Kept ingestion fast and stable under bursty sensor traffic, measured by zero dropped readings during spikes, by buffering incoming messages through an **SQS** queue ahead of processing. ↑ PERFORMANCE

> Built role-based auth across the dashboard and API. Hardened the app against **XSS/CSRF**, and **SSRF** — sanitizing/escaping rendered input, validating request origins, and restricting outbound requests from the server. ↑ SECURITY

Also designed the full architecture for a crypto payment service where each chain gets watched by its own container, which picks up transactions and sends webhooks to merchants — built with security in mind. On top of that sits a **GraphQL API** backed by **PostgreSQL**. Everything runs in **Docker** and gets built and deployed automatically from one infra repo that manages all the containers. Developed AI assistant bot with **RAG** and **LangChain** to help merchants with their questions and issues.

> Made each blockchain integration independently deployable and restartable, measured by one chain's outage never affecting the others, by running a dedicated watcher container per chain (**EVM**, **TRON**, **TON**, etc), orchestrated from a single infra repo with health checks and per-service memory limits. ↑ RELIABILITY

> Ensured merchant webhooks aren't lost or duplicated under load, measured by consistent delivery during chain re-syncs, by moving webhook dispatch into a dedicated worker backed by a **BullMQ**/**Redis** queue instead of firing requests inline. ↑ RELIABILITY

> Cut manual deploy steps to zero, measured by every push building and shipping on its own, by wiring up CI/CD that builds each service's Docker image and publishes it to **GHCR** for the whole container fleet. ↑ DEV SPEED

> Optimized AI bot under free-tier limits by implementing caching and using the **Grok** model, reducing redundant API requests and overall usage while maintaining a responsive user experience. ↓ COST

2022.07 - 2026.05

Kupsilla LLC

Biotech — cloud lab automation & genetic research

React · AngularJS · TypeScript · Sass · Storybook · Next.js · TanStack · Python · Java · GraphQL · AWS · Git

At Kupsilla I worked on a cloud chemical lab automation platform, and a human genetic research application.

Conducted **technical interviews** for junior and mid-level candidates, performed **code reviews**, and **mentored** developers to keep code quality high and support team growth.

Created a **GraphQL boilerplate** to fast develop MVPs of any complexity quickly with a focus on scalability. Its key feature is a single source of truth for all layers (backend, frontend, database/ORM, swagger/playground)

Worked with **Python** and **Java**. While they are not my primary langs, I have used them for application development, integrations, and maintaining existing projects.

Strateos – A **Cloud Lab Automation-as-a-Service** platform where users remotely run real **chemical reactions via robotic systems**.

> Improved scalability and independent deployability of a large React codebase, measured by separate release cycles per team, by re-architecting the frontend into **microfrontends** using **React**, **TypeScript**, **SCSS**, and **Storybook**. ↑ SCALABILITY

> Increased development speed across teams, measured by reduced duplication of UI work, by building shared UI libraries and publishing them to a private **NPM registry** used by all microfrontend modules. ↑ DEV SPEED

> Built a complex chemical reaction constructor — a heavy custom UI component that lets users visually design lab reactions — and packaged it as a standalone NPM module to keep the main app lightweight and maintainable. ↑ MAINTAINABILITY

GeneScience – A web application for searching, analyzing, and visualizing scientific parameters related to human genes. I designed the application architecture and developed complex data visualization components capable of handling large datasets and data-intensive rendering.

Built the application using **Next.js** with SSR, **React**, **TanStack Query**, **TanStack Table**, and **SVG**-based visualizations. Implemented efficient data loading and pagination, optimized database

queries and data access patterns, and improved rendering performance for complex visualizations and large datasets.

> Reduced initial page load time for data-heavy genomic pages by implementing **SSR** and enabling **gzip compression** on API responses. ↑ PERFORMANCE

> Kept AWS infrastructure costs within the free tier, measured by zero hosting spend, by deploying on **AWS Amplify** and managing build-minute usage within free quota limits. ↓ COST

> Built a complex interactive view combining a data plot and a collapsible rows table in one synchronized layout using **nivo.rocks**, improving data exploration for researchers. ↑ UX

> Enabled smooth collaboration across the team, measured by a consistent CI/CD flow for all developers, by setting up **AWS Amplify** with **Amazon Cognito** (Google OAuth), linting, testing, and a **GitHub branching policy** from scratch. ↑ TEAM FLOW

2021.05 - 2022.07

Strata K.K.

Video training platform

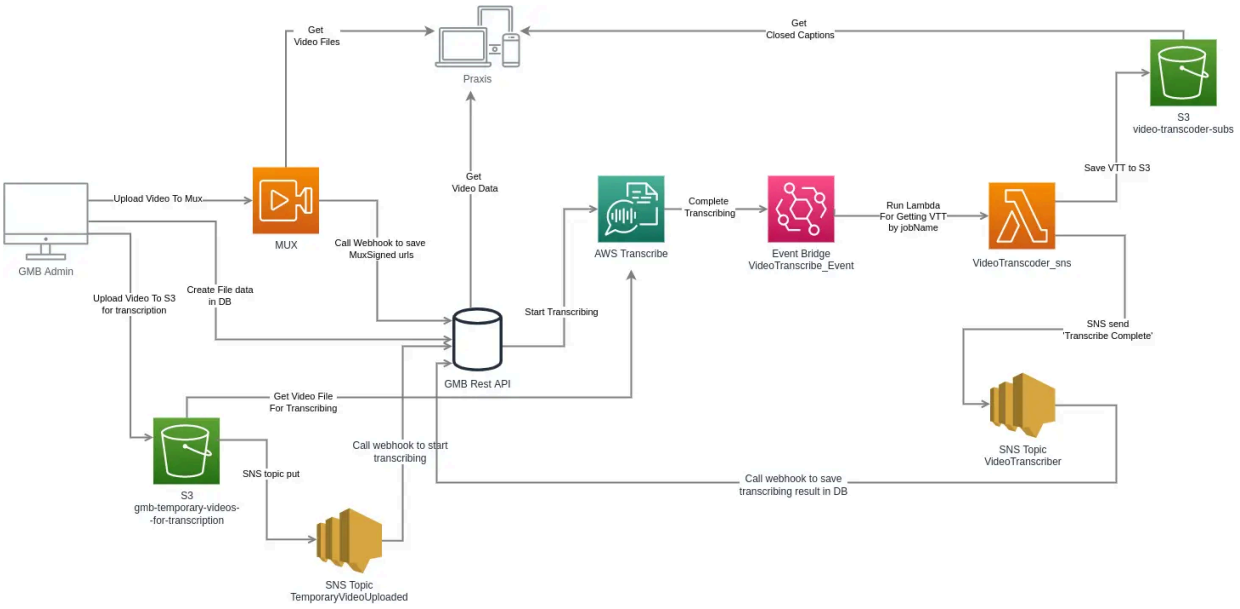
Next.js · NestJS · PostgreSQL · TypeORM · AWS · Auth0 · MUX

At Strata I worked on a **video training** platform for North American users. Stack: **Next.js**, **NestJS**, **PostgreSQL**, **TypeORM**, **Auth0**, **AWS Lambda**, **MUX**.

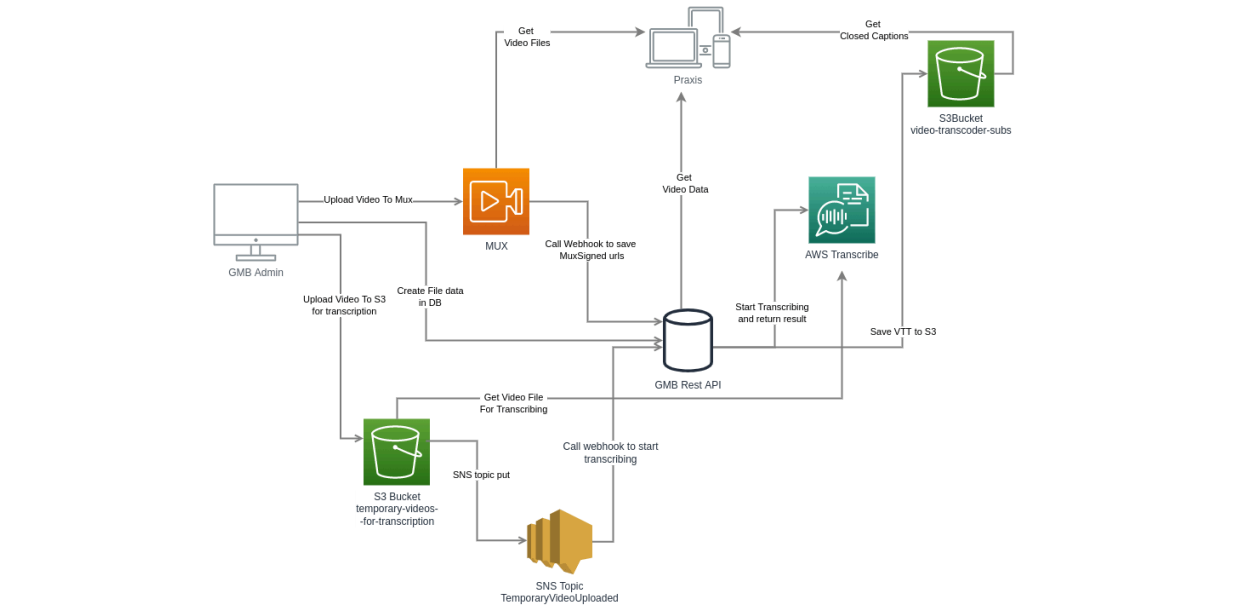
> Improved video streaming quality across a wide range of devices, measured by reduced buffering and format errors, by implementing automatic platform detection that selects the optimal video codec and resolution for each client. ↑ VIDEO QUALITY

> Reduced the maintenance cost of the video upload pipeline, measured by the number of AWS services needed to operate it, by simplifying the microservice architecture and removing redundant steps. ↓ OPS COST

PREVIOUS FLOW



SIMPLIFIED FLOW



Maintained and enhanced an admin panel built with **React Admin**, improving application performance and optimizing data-intensive workflows. Refactored existing code and architecture, upgraded core libraries and dependencies, resolved compatibility issues, and introduced improvements to maintainability, stability, and overall user experience.

2018.07 - 2021.04

Nwave Technologies Ltd.

IoT — smart parking sensors

AngularJS | React | TypeScript | AWS | Google Maps | Ionic | Capacitor

At Nwave, an IoT company building smart parking sensors for UK clients, I worked as a Full Stack Web Developer on both frontend and backend systems managing **20,000 IoT devices**.

I designed and developed a new REST API service for device management, using **PostgreSQL** on **AWS RDS**, **API Gateway**, **AWS Lambda**, **Amazon SNS**, **CloudWatch**, **Swagger/OpenAPI**, and automated client code generation. I designed the infrastructure and deployment pipeline using **AWS CDK** with **TypeScript**, supporting multiple environments and automated deployments. I also analyzed infrastructure costs and optimized the architecture using the AWS Pricing Calculator to reduce operational expenses.

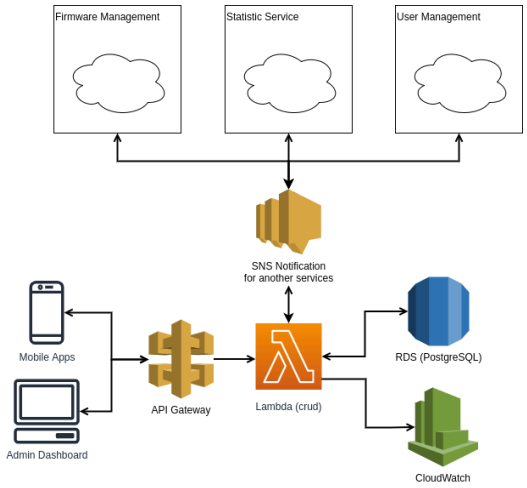
On the frontend, I developed an administrative dashboard for managing IoT devices and monitoring sensor statistics using **React**, **React Charts**, and **Google Maps**. I also upgraded the company's mobile application to the latest **Ionic** version and migrated the app to **Capacitor**.

> Built a **REST API** service handling full CRUD operations for 20,000 devices, achieving **99% test coverage**, by implementing it with **AWS Lambda**, **Node.js/TypeScript**, **PostgreSQL** (**PostGIS**), **AWS API Gateway**, and **AWS Cognito**, tested with **Mocha**, **Chai**, and the **AWS SDK**. ↑ RELIABILITY

> Improved infrastructure stability and enabled repeatable environment deployments, measured by one-command infrastructure setup, by adopting **AWS CDK** as infrastructure-as-code — replacing manual cloud configuration with version-controlled, reproducible stacks. ↑ DEVOPS

> Improved rendering performance of 20,000 map markers in the **SPlace PWA** mobile app, eliminating lag on low-end devices, by implementing marker clustering, removing unnecessary recalculations, and optimizing the **Ionic/Angular** rendering pipeline. ↑ PERFORMANCE

ARCHITECTURE DIAGRAM THAT I DEVELOPED



2017.05 - 2018.06

AdGuard

Ad blocker and browser extensions

JavaScript | Vue.js | Vuex | PostCSS

At AdGuard, one of the most popular ad blockers, I contributed to both the main product and additional browser extensions, written in **JavaScript (ES6)**:

- An extension that helps users read website content hidden behind ad-blocker detection walls (not deployed, available in a **repository**).
- Contributions to **AdGuard Assistant** (**repository**).

Also worked on the front-end of the **AdGuard** website using **Vue**, **Vuex**, and **PostCSS**.

2013.05 - 2017.05

Worked as a Junior and Middle Web Developer for several companies, building and maintaining websites using **PHP**, **WordPress**, **JavaScript**, and **jQuery**. Started working with modern JavaScript frameworks, including **Angular 1**, and built single-page applications. Gained experience with real-time features using **WebSockets** and created interactive **SVG** animations for various projects. Automated build and deployment processes using **Grunt** and **Gulp**, which improved development consistency across environments.